

Matteo Palmonari
matteo.palmonari@unimib.it



INSID&S Lab
*Interaction and Semantics for
Innovation with Data & Services*

2.4

Vocabularies for RDF

RDF: IRIfied Graphs and Triples

<dbr:Electric_Piano, rdfs:label, "Electric Piano"@en>
 <dbr:Electric_Piano, rdfs:label, "Piano Elettrico"@it>
 <dbr:Elton_John, dbo:instrument, dbr:Electric_Piano>
 <dbr:Empty_Sky, dbo:artist, dbr:Elton_John>
 <dbr:Sails, dbo:musicalArtist, dbr:Elton_John>
 <dbr:Sails, dbo:album, dbr:Empty_Sky>
 <dbr:Elton_John, dbo:instrument, dbr:Electric_Piano>
 <dbr:Elton_John, rdf:type, dbr:Musical_Artist>
 <dbr:Sails, rdf:type, dbo:Single>
 <dbr:Empty_Sky, rdf:type, dbo:Album>
 <dbo:Musical_Artist, rdfs:subClassOf, dbo:Artist>
 <dbo:Artist, rdfs:subClassOf, dbo:Person>

...

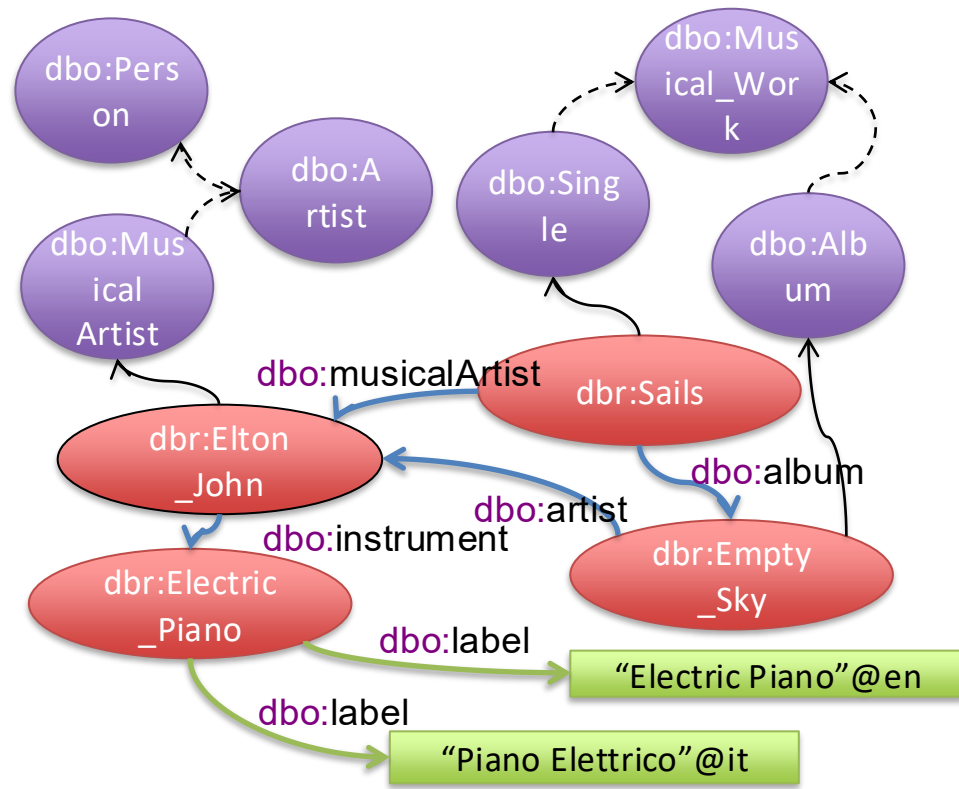
dbr: for <http://dbpedia.org/resource/>

e.g.: http://dbpedia.org/resource/Elton_John

dbo: for <http://dbpedia.org/ontology/>

rdfs for <https://www.w3.org/2000/01/rdf-schema#>

rdf for <http://www.w3.org/1999/02/22-rdf-syntax-ns#>



Categories by `rdf:type` assertions

Interpretation:

The resource in the *subject* belongs to the **category** specified by the *object*

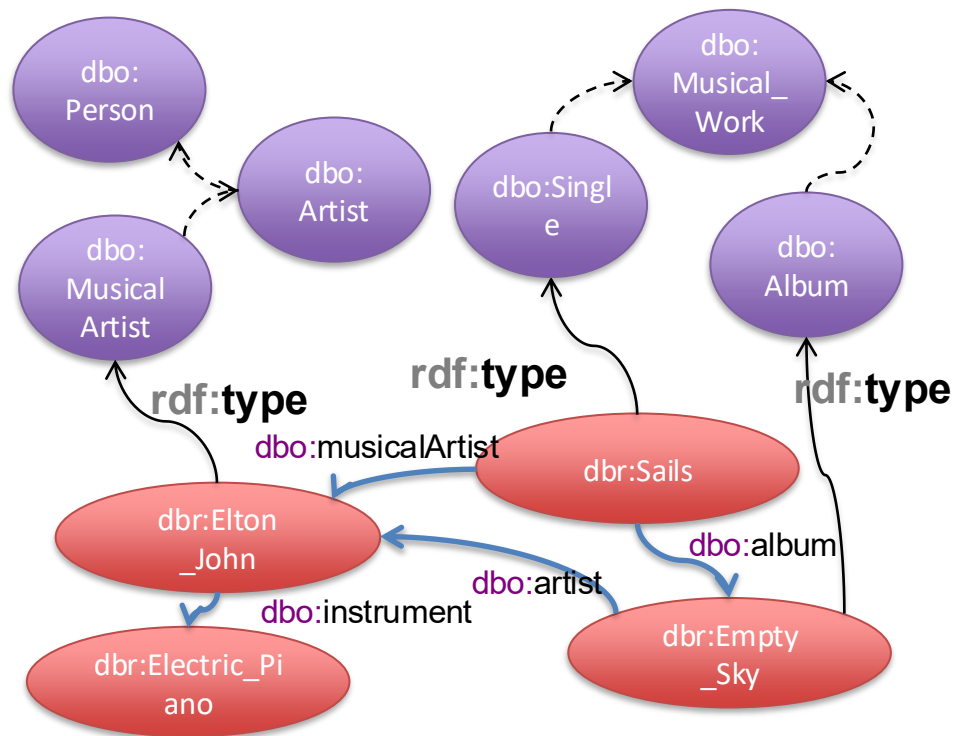
Examples:

`dbr:Elton_John` **`rdf:type`** `dbr:Musical_Artist` .

`dbr:Sails` **`rdf:type`** `dbo:Single` .

`dbr:Empty_Sky` **`rdf:type`** `dbo:Album` .

.... one of the very few predicates defined in the RDF specification itself



Data Schemas: **RDBs** vs RDF

■ Relational data (SQL): structured data

- Structure is given by the database schema (defined with the DDL)
 - Tables (entities described in the DB), attributes and their domains, relations, constraints
- Data and schema are **strongly coupled**
 - Schema-first: the schema is defined before entering the data
 - Prescriptive schema: data must be compliant with the schema to be inserted in the DB

```
CREATE TABLE Sells (  
    bar          CHAR(20),  
    beer         VARCHAR(20),  
    price REAL,  
    PRIMARY KEY (bar, beer)  
);
```

Data Schemas: RDBs vs RDF

■ RDF data: semi-structured data

- Some structure is given by the data model (triples)
- Data and schema are **loosly coupled**
 - Data must use URIs for properties and are expected to use URIs for classes → **vocabularies**
 - In principle... “data-first”: data can be inserted without any schema definition, what is used is the vocabulary (not a good practice: remember, one of our goal is use terms with shared meaning)
 - Non-prescriptive “schemas”: data are not required to be compliant with the schema to be inserted
 - Schema as “social agreement”: community-based definitions of recommended terms and usage
 - Deductive schema: the schema defines the meaning of the terms in the vocabulary in terms of inferences (e.g., triples added to the KG, inconsistencies)

Often referred
to as
“ontologies”

```
<dbr:Electric_Piano, rdfs:label, “Electric Piano”>
<dbr:Electric_Piano, rdfs:label, “Piano Elettrico”>
<dbr:Elton_John, dbo:instrument, dbr:Electric_Piano>
<dbr:Empty_Sky, dbo:artist, dbr:Elton_John>
<dbr:Sails, dbo:musicalArtist, dbr:Elton_John>
<dbr:Sails, dbo:album, dbr:Empty_Sky>
<dbr:Elton_John, dbo:instrument, dbr:Electric_Piano>
<dbr:Elton_John, rdf:type, dbr:Musical_Artist>
<dbr:Sails, rdf:type, dbo:Single>
<dbr:Empty_Sky, rdf:type, dbo:Album>
<dbo:Musical_Artist, rdfs:subClassOf, dbo:Artist>
<dbo:Artist, rdfs:subClassOf, dbo:Person>
...
dbr: for http://dbpedia.org/resource/
      e.g.: http://dbpedia.org/resource/Elton\_John
dbo: for http://dbpedia.org/ontology/
rdfs for https://www.w3.org/2000/01/rdf-schema#
rdf for http://www.w3.org/1999/02/22-rdf-syntax-ns#
```

Logic-based
languages
RDFS/OWL

Data Schemas: RDBs vs RDF with Example

■ Databases

□ Interpretation of schemas (example)

■ A *constraint-oriented* schema

- Song(ID, Title, Artist, Year)
- Artist(ID, Name, Label)
- Values for the attribute Artist in the Song table must be an (existing) ID in the Artist table
- Name in Artist must be UNIQUE

■ Implications

- If the ID does not exist in the Artist table, **data cannot be entered** in the Song table
- Information in the Artist table **must be filled in** completely (or NULL for non-key attributes)
- If a name is taken, it **cannot be reused** (e.g., "Placebo (2)")

■ RDF

□ Interpretation of schemas (example)

■ A *deductive* schema (simplified so far)

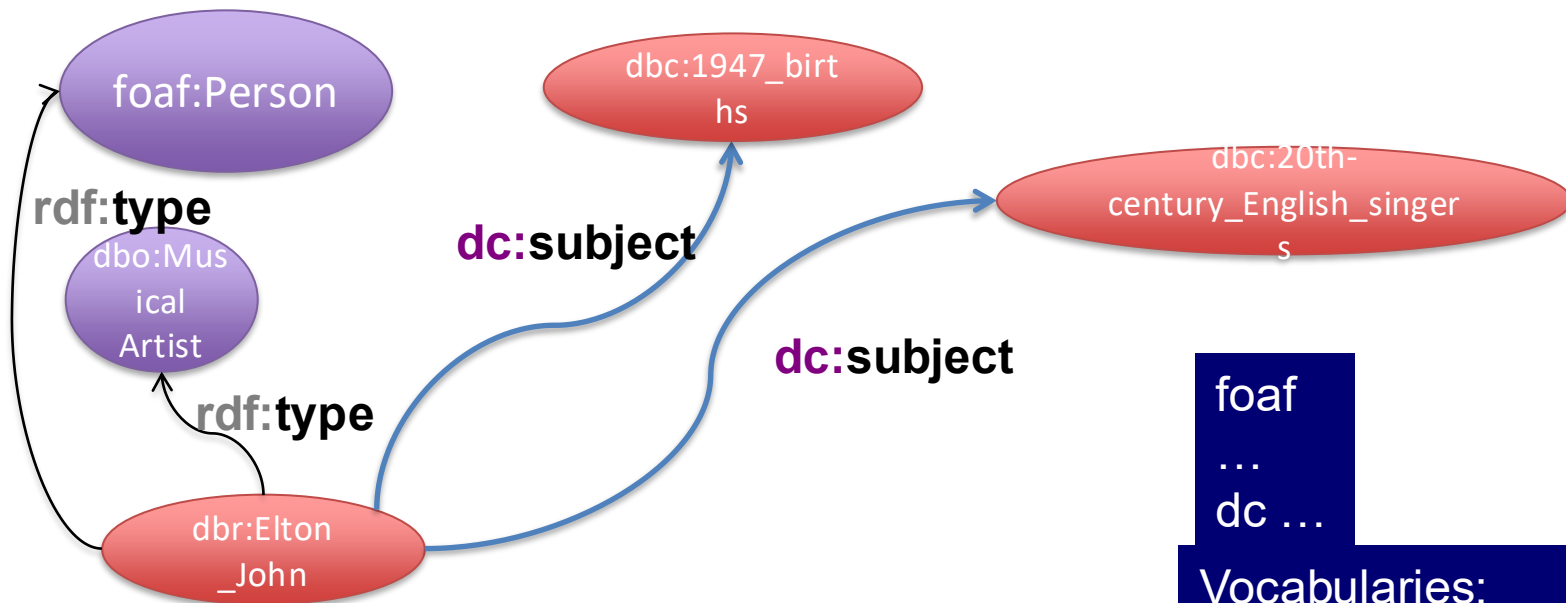
- Songs are artworks
- Songs' artists must be of type MusicArtist
- Values of year must be of type xsd:Year
- MusicArtists are Agents

■ Implications

- If **ex:s042** is a song, **then it is** an Artwork
- If **ex:a5946** is artist of some song, **then it is** a MusicArtist, **and also** an Agent
- If 2023 is a year of some song, **then it is** a xsd:Year
- In any case we can add a triple if it does not violate RDF model rules
- **A contradiction can be generated if** conflicting information is given, e.g., if **ex:a5946** is a Country, and countries are defined to be not Agensts

Constraints in terms of syntactic patterns and violations not part of deductive schema; can be defined/validated using SHACL Shapes Constraint Language

Shared Vocabularies with RDF



foaf

...

dc ...

Vocabularies:

- Properties
- Categories

...with shared
meaning

RDF in Action: FOAF



The *Friend of a Friend* (FOAF) project is creating a Web of machine-readable pages describing people, the links between them and the things they create and do

```
@prefix foaf: <http://purl.org/dc/elements/1.1/> .
```

```
<foaf:Person rdf:about="#danbri" xmlns:foaf="http://xmlns.com/foaf/0.1/">
```

```
<foaf:name>Dan Brickley</foaf:name>
```

```
<foaf:homepage rdf:resource="http://danbri.org/" />
```

```
<foaf:openid rdf:resource="http://danbri.org/" />
```

```
<foaf:img rdf:resource="/images/me.jpg" />
```

```
</foaf:Person>
```

example of FOAF

— metadata describing Dan Brickley

<http://www.foaf-project.org/>

<http://xmlns.com/foaf/spec/>

FOAF at a glance

An a-z index of FOAF terms, by class (categories or types) and by property.

Classes: | [Agent](#) | [Document](#) | [Group](#) | [Image](#) | [LabelProperty](#) | [OnlineAccount](#) | [OnlineChatAccount](#) | [OnlineEcommerceAccount](#) | [OnlineGamingAccount](#) | [Organization](#) | [Person](#) | [PersonalProfileDocument](#) | [Project](#) |

Properties: | [account](#) | [accountName](#) | [accountServiceHomepage](#) | [age](#) | [aimChatID](#) | [based_near](#) | [birthday](#) | [currentProject](#) | [depiction](#) | [depicts](#) | [dnaChecksum](#) | [familyName](#) | [family_name](#) | [firstName](#) | [fundedBy](#) | [geekcode](#) | [gender](#) | [givenName](#) | [givenname](#) | [holdsAccount](#) | [homepage](#) | [icqChatID](#) | [img](#) | [interest](#) | [isPrimaryTopicOf](#) | [jabberID](#) | [knows](#) | [lastName](#) | [logo](#) | [made](#) | [maker](#) | [mbox](#) | [mbox_sha1sum](#) | [member](#) | [membershipClass](#) | [msnChatID](#) | [myersBriggs](#) | [name](#) | [nick](#) | [openid](#) | [page](#) | [pastProject](#) | [phone](#) | [plan](#) | [primaryTopic](#) | [publications](#) | [schoolHomepage](#) | [sha1](#) | [skypeID](#) | [status](#) | [surname](#) | [theme](#) | [thumbnail](#) | [tipjar](#) | [title](#) | [topic](#) | [topic_interest](#) | [weblog](#) | [workInfoHomepage](#) | [workplaceHomepage](#) | [yahooChatID](#) |

PROV-O: The Provenance Vocabulary

<https://www.w3.org/TR/prov-o/>

Starting Point classes and properties provide the basis for the rest of the PROV Ontology and thus it is recommended that readers become comfortable with how to apply these terms before continuing to the remaining categories. These terms are used to create simple provenance descriptions that can be elaborated using terms from other categories. The classes and properties in this category are listed below and are discussed in [Section 3.1](#).

[prov:Entity](#) [prov:Activity](#) [prov:Agent](#)

[prov:wasGeneratedBy](#) [prov:wasDerivedFrom](#) [prov:wasAttributedTo](#)
[prov:startedAtTime](#) [prov:used](#) [prov:wasInformedBy](#) [prov:endedAtTime](#)
[prov:wasAssociatedWith](#) [prov:actedOnBehalfOf](#)

Expanded classes and properties provide additional terms that can be used to relate classes in the Starting Point category. The terms in this category are applied in the same way as the terms in the Starting Point category. Many of the terms in this category are subclasses or subproperties of those in the Starting Point category. The classes and properties in this category are listed below and are discussed in [Section 3.2](#).

[prov:Collection](#) [prov:EmptyCollection](#) [prov:Bundle](#) [prov:Person](#)
[prov:SoftwareAgent](#) [prov:Organization](#) [prov:Location](#)

[prov:alternateOf](#) [prov:specializationOf](#) [prov:generatedAtTime](#)
[prov:hadPrimarySource](#) [prov:value](#) [prov:wasQuotedFrom](#)
[prov:wasRevisionOf](#) [prov:invalidatedAtTime](#) [prov:wasInvalidatedBy](#)
[prov:hadMember](#) [prov:wasStartedBy](#) [prov:wasEndedBy](#) [prov:invalidated](#)
[prov:influenced](#) [prov:atLocation](#) [prov:generated](#)

As used at Amazon...

It does scale! Here is a great example...

Inventory knowledge graph of Amazon Fulfillment is used to

- investigate fulfillment processes
- investigate lost & found -issues
- improve precision of product recalls

Extends the PROV-O ontology:
models the end-to-end logistics
process as a form of provenance

Runs on multiple (federated)
Neptune clusters

Size:

1T+ triples

4B new triples per day

Queries (p95):

< 50 ms to find a node

< 1 s to retrieve the whole path

From [Ora Lassila's keynote at ISWC 2024](#)

Principal Technologist in the Amazon Neptune graph database team
and the co-chair of the W3C RDF-star Working Group

Vocabularies as Social Agreements

- A group of persons can agree to use a specific **vocabulary**; in RDF:

- **Classes** to type resources, e.g., [foaf:Document](#)
- **Properties** to describe resources, e.g., [foaf:topic](#)

- **Prescriptions / recommendations** on how to use the properties:

- which types of subjects, e.g. use [foaf:topic](#) to describe resources of type [foaf:Document](#) or [foaf:Image](#)
- which types of objects, e.g., the objects for [foaf:made](#) should not be a literal

Documents and Images

- [Document](#)
- [Image](#)
- [PersonalProfileDocument](#)
- [topic](#) ([page](#))
- [primaryTopic](#) ([primaryTopicOf](#))
- [tipjar](#)
- [sha1](#)
- [made](#) ([maker](#))
- [thumbnail](#)
- [logo](#)

Schema.org

- “a collaborative, community activity with a mission to create, maintain, and promote schemas for structured data on the Internet, on web pages, in email messages, and beyond.”
 - can be used with many different encodings, including RDFa, Microdata and JSON-LD
 - 10+M websites using it
 - Collaborative effort by big internet players (Google, Microsoft, Yahoo and Yandex)
 - Evolving along time
- Visit and browse at: <https://schema.org/>

Example - Schema.org:Event

schema.org

Custom Search



Home

Schemas

Documentation

Event

[Thing](#) > [Event](#)

An event happening at a certain time and location, such as a concert, lecture, or festival. Ticketing information may be added via the [offers](#) property. Repeated events may be structured as separate Event objects.

[\[more...\]](#)

Property	Expected Type	Description
Properties from Event		
about	Thing	The subject matter of the content. Inverse property: subjectOf .
actor	Person	An actor, e.g. in tv, radio, movie, video games etc., or in an event. Actors can be associated with individual items or with a series, episode, clip. Supersedes actors .
aggregateRating	AggregateRating	The overall rating, based on a collection of reviews or ratings, of the item.
attendee	Organization or Person	A person or organization attending the event. Supersedes attendees .
audience	Audience	An intended audience, i.e. a group for whom something was created. Supersedes serviceAudience .
composer	Organization or Person	The person or organization who wrote a composition, or who is the composer of a work performed at some event.

FOAF

Primitives



What is a Person?

FOAF Basics

- Agent
- Person
- name
- nick
- title
- homepage
- mbox
- mbox_sha1sum
- img
- depiction (depicts)
- surname
- familyName
- givenName
- firstName
- lastName

Personal Info

- weblog
- knows
- interest
- currentProject
- pastProject
- plan
- based_near
- age
- workplaceHomepage
- workInfoHomepage
- schoolHomepage
- topic_interest
- publications
- geekcode
- myersBriggs
- dnaChecksum

Online Accounts / IM

- OnlineAccount
- OnlineChatAccount
- OnlineEcommerceAccount
- OnlineGamingAccount
- account
- accountServiceHomepage
- accountName
- icqChatID
- msnChatID
- aimChatID
- jabberID
- yahooChatID
- skypeID

How can we specify the meaning of types and predicates?

A **vocabulary** (a list of allowed terms) is not enough...

In **Ontologies** we use axioms to constrain the meaning of terms in a way that is useful for a machine, i.e., in a way that even a machine can make inferences on top of the data

Ontologies

ontology noun



Save Word

on·tol·o·gy | \ ăn-'tă-lə-jē \



Definition of *ontology*

- 1 : a branch of metaphysics concerned with the nature and relations of being
// *Ontology* deals with abstract entities.
- 2 : a particular theory about the nature of being or the kinds of things that have existence

Ontologies in Information Sciences by Tom Gruber

- An ontology is a description (like a formal specification of a program) of the **concepts** and **relationships** that can formally exist **for an agent or a community of agents**. This definition is consistent with the usage of ontology as set of concept definitions, but more general. And it is a different sense of the word than its use in philosophy.
- [...] To specify a conceptualization, one needs to state axioms that do constrain the possible interpretations for the defined terms**

* Gruber, T. (2001). "What is an Ontology?". Stanford University.

** Gruber, T (1993). "A translation approach to portable ontology specifications". Knowledge Acquisition. 5 (2): 199–220.

(Axiomatic) Ontologies

An **axiomatic ontology** is a formal specification of a conceptualization by means of **a language L** , a set of **logical axioms of L** and a formal semantics that uniquely determines the **meaning** of the terms, i.e., the symbols of L .

To determine the meaning of a term $\rightarrow$ to specify its unambiguous interpretation within a mathematical model



Axiomatic ontologies organize knowledge by defining:

- **Symbols** to represent **individuals** (e.g., entities like *Elton John*), **classes** (e.g., entity types like *Musical Artist*), **relations** (e.g., properties to describe entities like *made*) – **the vocabulary**
- **Axioms** to represent the logical constraints that specify their meaning and thus support **automated reasoning (inference)**

Reminder: Reasoning and Inference

rea·son·ing | \ 'rēz-niŋ , 'rē-zən-iŋ \

Definition of *reasoning*

1 : the use of reason

especially : the drawing of inferences or conclusions through the use of reason

2 : an instance of the use of reason : ARGUMENT

■ Inferences / conclusions

- An inference establishes a relation

- Premises → Conclusion

- Computational models of inference

- Which conclusions can we draw from a set of premises?

- Why is the conclusion correct?

Reminder: Reasoning and Inference

- Goal: model agents that can manipulate knowledge and execute knowledge-intensive tasks
 - Facts + general domain knowledge
 - {Paris is located in France, France is located in Europe} + {is located in is a transitive relation} $\subseteq$ KB
 - Reasoning: inferences and inference-aware queries
 - $\text{KB} \vdash \{\text{Paris is located in Europe}\}$
 - Where is Paris located? {France, Europe}
 - ... more on this when we introduce ontologies 😊

Some Remarks on Schemas/Ontologies

- More on: ontologies vs database schemas
 - Database schemas: several technical details (e.g., length of strings)
 - Ontologies: modeling is more conceptual (e.g., all musicians are persons, persons cannot be friend of places, etc.)
 - Technical data modeling for RDF: Shapes Constraint Language (SHACL)
 - W3C recommendation from 2017: <https://www.w3.org/TR/shacl/>
 - Detailed constraints on the data
 - Specific engines can validate the data (data quality)
- Property-graph data model
 - Also: semi-structured, loose schema/data coupling
 - Usually schema is somehow defined: node types, relationship types, etc.

Recap

- RDF: a data model for publishing data on the web
 - Triples using IRIs + literals + blank nodes
- RDF KGs: “linked data”, optimized for knowledge sharing on the web
- Vocabularies: organization of knowledge to support sharing and interoperability
- Highly recommended: explore DBpedia and Wikidata